WarPuppy Platform & Project Portfolio

A Technical Overview of the Hermes Agent Environment and the Systems Built On It

---

Prepared for **Johann (WarPuppy / GreenyZA)**
Technical overview — platform capability and project inventory
2026-10-02

29 projects · 115 agent skills · 3-tier local-first model routing
Lenovo M710Q “anvil” · Acer GX-281 “hearth”

Executive Summary

This document explains what has been built on this workstation and what the underlying platform — Hermes Agent — is capable of. It is written so that a technically literate reader who has never seen this machine can understand the scope, architecture, and intent of the work.

The environment is a **self-hosted, local-first AI agent platform** running on Arch Linux across two machines on a home LAN (Lenovo "anvil" as the local shell, Acer "hearth" as the server/GPU node). On top of that platform, roughly **29 distinct projects** have been developed, spanning:

- A **multi-tier LLM routing and token-optimisation system** that keeps AI work on local GPU and free cloud tiers before ever touching a paid API.
- A **governance layer** (Board of Advisors + ITIL) that decides how much effort and how many agents a task deserves.
- A **business platform** (Clarity) with a public brand, an operations dashboard, legal/POPIA compliance documents, a customer pipeline, and a documented harassment-investigation product.
- Several **engineering projects**: a native C++/Vulkan orbital-visualisation engine (Phosphor) and its Rust successor (RustyPhosphor), a Linux-native drone research programme (Spark), a two-node workstation cluster (Twinforge → Stout Linux), and an observability stack.
- A **multi-agent orchestration model** with workers, reviewers, and a top-level approver.

The through-line is **local-first economics with governed autonomy**: expensive models and cloud spend are treated as a last resort, and non-trivial work is automatically parallelised, reviewed, and approved by specialist agents rather than a single model acting alone.

Part 1 — The Platform: Hermes Agent

1.1 What Hermes Agent Is

Hermes Agent is the terminal-based AI agent harness this environment runs on. It is not a chatbot — it is an **agentic runtime**: it can read and write files, run shell commands, drive a real browser, search the web, call external APIs, schedule background jobs, spawn sub-agents, and maintain persistent memory across sessions.

Property	Value
Version	v0.21.5+5727 (2026.9.24)
Install method	git
Runtime Python	3.14.7
Interface	TUI (terminal UI), plus desktop and CLI modes
Configuration root	`~/.hermes/`
Persistent memory	`~/.hermes/memories/` (per-profile)
Session store	`~/.hermes/sessions/` + `state.db`
Skills library	115 skills installed

1.2 Core Capabilities

Tooling. The agent has a broad, composable toolset:

- **Filesystem** — read, write, and targeted patched edits with fuzzy matching and syntax checking.
- **Terminal** — a persistent shell session with foreground and background processes, PTY support, and output capture.
- **Code execution** — a persistent Python kernel for multi-step logic, filtering, and loops (up to 50 tool calls per run).
- **Web** — search plus clean page extraction, with PDF support.
- **Browser** — a real browser it can drive end-to-end (navigate, fill forms, click, screenshot), including a credential vault for logins and payments.
- **Vision** — it can look at images and answer questions about them.
- **Sub-agents** — parallel `delegate_task` workers in isolated contexts.
- **Scheduling** — cron jobs for recurring work.
- **Memory** — durable facts that survive across sessions.

Skills. The platform has a **skills system** — reusable procedural knowledge packages loaded on demand. A skill encodes *how* a recurring task is done in this environment: the exact commands, the pitfalls, the user's preferred approach. With 115 skills installed, the agent does not have to re-derive workflows; it loads the relevant one. Skills are organised by category (creative, devops, github, mlops, productivity, research, software-development, and more) and can be created, patched, and versioned by the agent itself.

Profiles. Multiple isolated agent profiles can coexist, each with its own skills, plugins, cron jobs, and memory — so different identities or use-cases do not contaminate each other.

Persistence and recovery. Conversation history is stored in SQLite with full-text search, so past sessions can be searched and resumed. Context compression keeps long sessions viable. Emergency backups of the state database are taken automatically before updates.

1.3 The Governance Layer (BOA + ITIL)

The distinctive feature of this environment is that it does not simply "answer" — it **governs** how work is done.

- **Token Optimiser (TOS)** scores every task for a cheapest-viable route, with per-category confidence thresholds. Security-sensitive work is capped (e.g. 55% confidence) and escalated to human review rather than auto-executed.
- **ITIL classification** runs first on operational questions (incident / problem / change / request / event), so the agent knows what *kind* of work it is handling before it starts.
- **Board of Advisors (BOA)** — six seats (Architect, Engineer, Researcher, Reviewer, Budget, Executive) — reviews the routing and swarm decisions for non-trivial work.
- **Swarm validation** decides whether a task is solo, a pair, or a full swarm, based on file count, module count, and sequential-tool-call thresholds.

The result: the same governance that a consultancy would apply to project intake is applied automatically to each task, and free/local tiers are maximised before paid models are used.

1.4 The Model Tier System

AI work is routed across three tiers:

Tier	Provider	Examples	Role
T1	Local llama.cpp on Vulkan (`:8081`)	Qwen2.5-3B, Phi-3.5-mini	Cheap grunt work, code, formatting
T1b	Lenovo M710Q CPU inference	Qwen2.5-7B	Secondary local capacity
T2	Free cloud (OpenRouter, NVIDIA NIM)	40+ free models	Research, documentation, general tasks
T3	Paid / premium (escalation only)	Grok, heavyweight models	Hard jobs, final writing

The design principle is explicit: **T1 and T2 before T3.** Paid models are reserved, not default.

1.5 Hardware Footprint

Node	Hardware	Role
Lenovo M710Q ("anvil")	Arch Linux / Omarchy, Intel HD 630 iGPU	Local agent shell, development, helper node
Acer GX-281 ("hearth")	Arch Linux, AMD RX 480 (4 GB, Polaris)	GPU inference, Docker services, observability, canonical file store

---

Part 2 — The Flagship System: neo-light

2.1 Purpose

neo-light is the packaged, publishable form of this entire environment: a lightweight agent environment with BOA/ITIL-governed token-optimiser routing, automatic swarming, and a 3-tier budget that prioritises local and free resources.

It is designed to be cloned and run by anyone with a Python 3.10+ machine — the same economics that make this environment cheap to operate are made portable.

2.2 Repository Family

The system is deliberately split so that a shared core serves multiple user forks:

Repository	Role
neo-light-core	Shared library: router, BOA, config, mock backends, dashboard, 78 integration tests
neo-light	Main environment (the canonical/public repo)
neo-light-warpuppy	User fork — pins core as a git submodule, carries local data
neo-light-phantom	Cloud-only variant (no GPU): NVIDIA-first, Grok media models
light-engine	The original workspace that neo-light was extracted from

The submodule pattern means the core is versioned once (semver tags) and every fork consumes it — no duplicated logic, no drift.

2.3 Core Library Components

File	Responsibility
`config.py`	TokenOptimiserConfig schema and defaults
`router.py`	TokenRouter, ITIL classification, parallelise/swarm scoring
`boa.py`	Board of Advisors (6 seats), governance, TOS probing, swarm validation
`monica_integration.py`	`optimise()` wrapper, calibration, learning log
`mock_backends.py`	T1/T2/T3 mock backends + test harness
`dashboard/`	FastAPI dashboard for live config and analytics
`tests/`	78 integration tests

2.4 Swarm-First Delegation

The system enforces a swarm-first pattern for multi-file, multi-module work:

1. **Orchestrator** decomposes the task into independent sub-tasks.
2. **Workers** execute in parallel (one per sub-task).
3. **Reviewers** validate each worker's output independently.
4. Work is only marked done when all reviewers issue a clean pass (zero Critical/High findings).

This triggers automatically for any task touching more than 3 files, more than 2 modules, or requiring more than 5 sequential tool calls. The intent is that quality does not depend on a single model's self-assessment — **no agent approves its own work.**

Part 3 — Project Inventory

The following is a complete inventory of the projects built on this platform, grouped by domain.

3.1 Clarity — Business Platform

Clarity is a self-hosted, AI-managed business platform for training, AI consulting, and IT services, all under one brand and one domain (**goclarity.co.za**), running on the household server.

Three service pillars:

1. **Clarity Learn** — Training & Upskilling (1-on-1, group, AI training, session packages)
2. **Clarity AI** — AI Consulting & Services (strategy, readiness audit, integration, POC)
3. **Clarity Consult** — IT Consulting & Service Desk (general IT, remote sysadmin, managed services)

Project	Description	Status
ClarityLearn	The full platform: website, dashboard, legal docs, templates, service menu, backup system, second-brain wiki	Active
Clarity (teaser)	Public teaser site for goclarity.co.za	Published

Project	Description	Status
clarity-site	Fuller static site (home, offer, refuse, ask pages, browser-only Q&A from `knowledge.json`)	Parked
clarity-prime	Application of the Grok Build Master Prime guide — vault structure, constitution, project stubs	Complete
clarity-phone	Google account bridge: Calendar, Gmail, Tasks, Contacts, Drive from the Android phone	Phase 0
clarity-video	Faceless video production pipeline (script → cards → silent MP4 via ffmpeg)	Parked
clarity-wiki	Read-only LAN web view of the Clarity vault (Perlite + nginx on `:3020`)	Live
clarity-token	ClarityToken — non-forgeable intake token + harassment-investigation product	Active

Notable sub-systems:

- **Analytics engine & dashboard** — central business dashboard with overview, tools, analytics, pillars, operations, settings, and logs.
- **POPIA compliance suite** — privacy policy, PAIA manual, terms of service, refund policy, consent forms, compliance checklist, and per-pillar service agreements.
- **Second Brain** — an Obsidian vault following the Karpathy LLM Wiki pattern: raw input in `raw/`, AI-maintained summaries and cross-references in `wiki/`, with strong wiki-style linking.
- **Backup system** — local + Google Drive backup, sync, and weekly verification scripts.

ClarityToken deserves specific mention. It is a complete product workflow where each engagement is one token that ties together onboarding → token minting → victim intake (PII encrypted at rest, never over DM) → X/Twitter investigation → report → invoice. The invoice is appended to a **tamper-evident hash chain** alongside the job cards, so the invoice provably matches the work done.

3.2 Agent Environment & Orchestration

Project	Description
neo-light	The flagship agent environment (see Part 2)
neo-light-core	Shared routing/BOA library
neo-light-warpuppy	User fork
neo-light-phantom	Cloud-only variant

Project	Description
light-engine	Original Neo workspace (source of neo-light)
multi-agent	Matrix-themed switchable swarm: Neo (orchestrator), Tank/Sentinel (bash), Switch/Link (markdown), Mouse/Keymaker (yaml), Trinity/Niobe (llama.cpp Vulkan), Morpheus/Ghost (ssh), Apoc/Dozer (model routing), The Architect (final approval)
pinky	A 30-agent multi-agent operating system for the Hermes codebase: 1 orchestrator (Monica), 14 specialist workers, 14 paired reviewers, 1 top-level approver (Omega) — strict worker → reviewer → approver pipeline
pinky-optimiser	Token optimiser developed alongside pinky
token-optimiser	The original Lenovo-adapted token optimiser (OpenRouter → NVIDIA → Grok last)
work/boa	Board of Advisors governance work area

The multi-agent model is the same everywhere: **workers never self-approve, reviewers never fix, and the top-level approver only reviews the integrated whole.**

3.3 Inference & Infrastructure

Project	Description
llama-cpp-vulkan	GPU inference on the Acer RX 480: `llama-cli` at 106 t/s on Qwen2.5-3B Q4_K_M, systemd `llama-server` on `:8081`, documented in HOST.md and SMOKE.md
acer-observability	Prometheus + Grafana + cAdvisor stack on the Acer: Grafana `:3001`, Prometheus `:9090`, Portainer `:9443`. Scrapes host node_exporter, llama-server metrics, cAdvisor
twinforge	Two-node Arch workstation cluster (Lenovo desk + Acer GPU). Copper gigabit, distcc, Input Leap KVM, Wake-on-LAN, helper mirror. Not one OS — a shop-floor cluster
stout	Stout Linux — a personal OS project tree (source, not an installed OS). Twinforge is the seed; Stout is the end state. Built in QEMU with pacman

3.4 Engineering & Visualisation

Project	Description
orbit-plot (Phosphor)	Native C++ desktop orbit-visualisation tool. Retro phosphor wireframe with modern satellite catalog. Three cameras (celestial, earth, helio), Vulkan 1.2, GPU SGP4 propagation, 10k+ satellites from CelesTrak, Dear ImGui UI
rusty-phosphor	Full Rust rewrite of Phosphor as a 10-crate Cargo workspace: Vulkan renderer, 100k+ stars in a single draw call, 30k+ satellites, scanline/bloom/vignette/chromatic-aberration post-processing, egui interface
spark	DJI Spark drone project: Linux-native operation research (DUML protocol, telemetry, ground-station bridge), a professional research report, an LLM Wiki v2 knowledge base, and an investor-ready business plan for a rural drone inspection & security service
sprite_humanoid_freecad	FreeCAD hardware design source for the Sprite humanoid robot (third-party, CERN-OHL-S-2.0 licensed)

Phosphor is the deepest engineering effort: a from-scratch rendering engine where every feature is built on the engine rather than beside it. The plan is explicitly engine-first — the scene graph, instanced renderer, and post-processing chain are written once and shared by all three camera modes, developed on the Lenovo iGPU and validated on the Acer RX 480.

Spark is the most commercially developed: it pairs hard technical reverse-engineering research with a full business case (financial model, sensitivity analysis, regulatory pathway under SACAA Part 101, go-to-market via local networks).

3.5 Knowledge, Media & Tooling

Project	Description
gstack	Garry Tan's open-source Claude Code skill pack (third-party clone, MIT) — a virtual engineering team of 23 specialists as slash commands
olivia	Customer prompt material
olivia-x	Investigation evidence sets and voice note (customer C-001)
work/	Working directory: MAC-Telnet, blender, blender-skills-packs, GTNH Minecraft, tiny-distro

3.6 The Clarity Vault (Obsidian)

Underpinning the business work is a structured Obsidian vault at `~/Documents/Clarity`, organised into numbered folders:

- `00-Inbox`, `01-Projects`, `02-Skills`, `03-Knowledge-Base`, `04-Constitutions`, `05-Prompts-Library`, `06-Research-Loops`, `07-Video-Production`, `08-Website-Build`

It contains the constitution, the offer definition, customer records, a Board of Advisors session archive, a **PROJECT_LEDGER.md** (append-only decision log with rationale, route, and rollback for every significant decision), and a restore map (`INDEX.md`) that documents how the whole system is recovered. It is mirrored hourly to the Acer and versioned on GitHub.

Part 4 — Cross-Cutting Design Principles

Several principles recur across every project, and they explain why the system is built the way it is:

1. Local-first, free-first economics. Local GPU and free cloud tiers are used before paid APIs. This is not a preference — it is enforced in code by the token optimiser.

2. Governed autonomy. Non-trivial work is classified (ITIL), scored (BOA), and sized (solo/pair/swarm) before it runs. Autonomy is real but bounded.

3. No self-approval. Workers never approve their own work. Every change passes a paired reviewer, and integrated changes pass a top-level approver.

4. Append-only decision records. The project ledger and wiki log record not just *what* was decided but *why*, with rollback paths — so the system is recoverable and auditable.

5. Knowledge compounding. The LLM Wiki pattern (raw → wiki, with confidence scoring and cross-linking) means research findings compound instead of being re-derived.

6. Documentation as a first-class artifact. Every project carries a README, a PLAN with checkboxes, and where relevant a professional report. Plans are living documents.

7. Open and portable. The core is a shared library; user data is separate. Nothing depends on a closed ecosystem — a deliberate reaction against the vendor lock-in that motivated the Spark drone research in the first place.

Part 5 — What the Environment Can Do Today

For a reader trying to gauge capability rather than inventory, this is what the platform does end-to-end, right now:

- **Take a complex task and route it.** Score it, classify it, decide the tier, decide the swarm size, and execute — with free/local resources first.
- **Run work in parallel.** Spawn multiple isolated sub-agents, each with its own context and toolset, and have their output independently reviewed.
- **Operate the machine.** Read/write files, run commands, manage background processes, install software, configure services.
- **Operate the web.** Search, extract, and drive a real browser end-to-end — including authenticated sessions.
- **Produce deliverables.** Reports, PDFs, spreadsheets, documents, presentations, diagrams, video, code — generated, not described.
- **Persist and recall.** Remember durable facts across sessions and search the full history of past conversations.
- **Schedule itself.** Run recurring jobs on a cron.
- **Govern itself.** Apply the same intake and review discipline to its own work that a professional services firm would apply to a project.
- **Extend itself.** Create and update its own skills, so the environment gets better at recurring work over time.

Appendix A — Project Index

#	Project	Domain	Status
1	ClarityLearn	Business platform	Active
2	Clarity (teaser)	Public brand	Published
3	clarity-site	Public site	Parked
4	clarity-prime	Vault / constitution	Complete
5	clarity-phone	Google bridge	Phase 0
6	clarity-video	Media pipeline	Parked
7	clarity-wiki	LAN wiki	Live
8	clarity-token	Investigation product	Active
9	neo-light	Agent environment	Active
10	neo-light-core	Shared library	Active
11	neo-light-warpuppy	User fork	Active
12	neo-light-phantom	Cloud variant	Active
13	light-engine	Original workspace	Active

#	Project	Domain	Status
14	multi-agent	Matrix swarm	Active
15	pinky	30-agent OS	Complete
16	pinky-optimiser	Token optimiser	Complete
17	token-optimiser	Original optimiser	Complete
18	llama-cpp-vulkan	GPU inference	Live
19	acer-observability	Monitoring stack	Live
20	twinforge	Two-node cluster	Phases 0–7
21	stout	Personal OS	Early
22	orbit-plot (Phosphor)	C++ orbit engine	Phases G0–G12
23	rusty-phosphor	Rust orbit engine	Active
24	spark	Drone research + business	Parked
25	sprite_humanoid_freecad	Robot CAD	Third-party
26	gstack	Skill pack	Third-party
27	olivia / olivia-x	Customer material	Archive
28	work/ (MAC-Telnet, blender, gtnh, tiny-distro)	Tooling	Mixed

Appendix B — Host & Environment Reference

Item	Value
Local shell	Lenovo M710Q — Arch Linux / Omarchy
Server	Acer GX-281 — Arch Linux, AMD RX 480 4 GB
GPU inference endpoint	`http://192.168.1.10:8081/v1` (llama.cpp Vulkan)
Agent config root	`~/.hermes/`
Installed skills	115
Projects root	`~/Projects/`
Vault	`~/Documents/Clarity`
Business domain	goclarity.co.za
GitHub org	github.com/GreenyZA

End of document.